

jq80qcj7o

April 26, 2025

```
[57]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[88]: import zipfile

zip_path = '/content/drive/MyDrive/archive.zip'
extract_path = '/content/data'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
```

```
[59]: import numpy as np
import pandas as pd

import os
for folder_path, _, file_list in os.walk('/content/data/fer2013/fer2013/fer2013.
↪csv'):
    for single_file in file_list:
        print(os.path.join(folder_path, single_file))
```

```
[89]: pip install tensorflow
```

Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (25.2.10)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in

/usr/local/lib/python3.11/dist-packages (from tensorflow) (18.1.1)
 Requirement already satisfied: opt-einsum>=2.3.2 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.4.0)
 Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (25.0)
 Requirement already satisfied:
 protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3
 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (5.29.4)
 Requirement already satisfied: requests<3,>=2.21.0 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.32.3)
 Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (75.2.0)
 Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (1.17.0)
 Requirement already satisfied: termcolor>=1.1.0 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.0.1)
 Requirement already satisfied: typing-extensions>=3.6.6 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (4.13.2)
 Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (1.17.2)
 Requirement already satisfied: grpcio<2.0,>=1.24.3 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.71.0)
 Requirement already satisfied: tensorboard~=2.19.0 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.19.0)
 Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (3.8.0)
 Requirement already satisfied: numpy<2.2.0,>=1.26.0 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.26.4)
 Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow) (3.13.0)
 Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.5.1)
 Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
 /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.37.1)
 Requirement already satisfied: wheel<1.0,>=0.23.0 in
 /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow)
 (0.45.1)
 Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages
 (from keras>=3.5.0->tensorflow) (14.0.0)
 Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages
 (from keras>=3.5.0->tensorflow) (0.0.9)
 Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages
 (from keras>=3.5.0->tensorflow) (0.15.0)
 Requirement already satisfied: charset-normalizer<4,>=2 in
 /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow)
 (3.4.1)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-
 packages (from requests<3,>=2.21.0->tensorflow) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.1.31)

Requirement already satisfied: markdown>=2.6.8 in /usr/lib/python3/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.3.6)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard~=2.19.0->tensorflow) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.1.3)

Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.2)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)

```
[90]: import math
import numpy as np
import pandas as pd

import seaborn as sns
from matplotlib import pyplot

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report

import tensorflow as tf
from tensorflow.keras import optimizers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Conv2D, MaxPooling2D
from tensorflow.keras.layers import Dropout, BatchNormalization, LeakyReLU,
↳Activation
from tensorflow.keras.callbacks import Callback, EarlyStopping,
↳ReduceLROnPlateau
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
[62]: data_frame = pd.read_csv('/content/data/fer2013/fer2013/fer2013.csv')
      print(data_frame.shape)
      data_frame.head()
```

```
(35887, 3)
```

```
[62]:
```

	emotion		pixels	Usage
0	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121...	Training	
1	0	151 150 147 155 148 133 111 140 170 174 182 15...	Training	
2	2	231 212 156 164 174 138 161 173 182 200 106 38...	Training	
3	4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 1...	Training	
4	6	4 0 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84...	Training	

```
[63]: data_frame.emotion.unique()
```

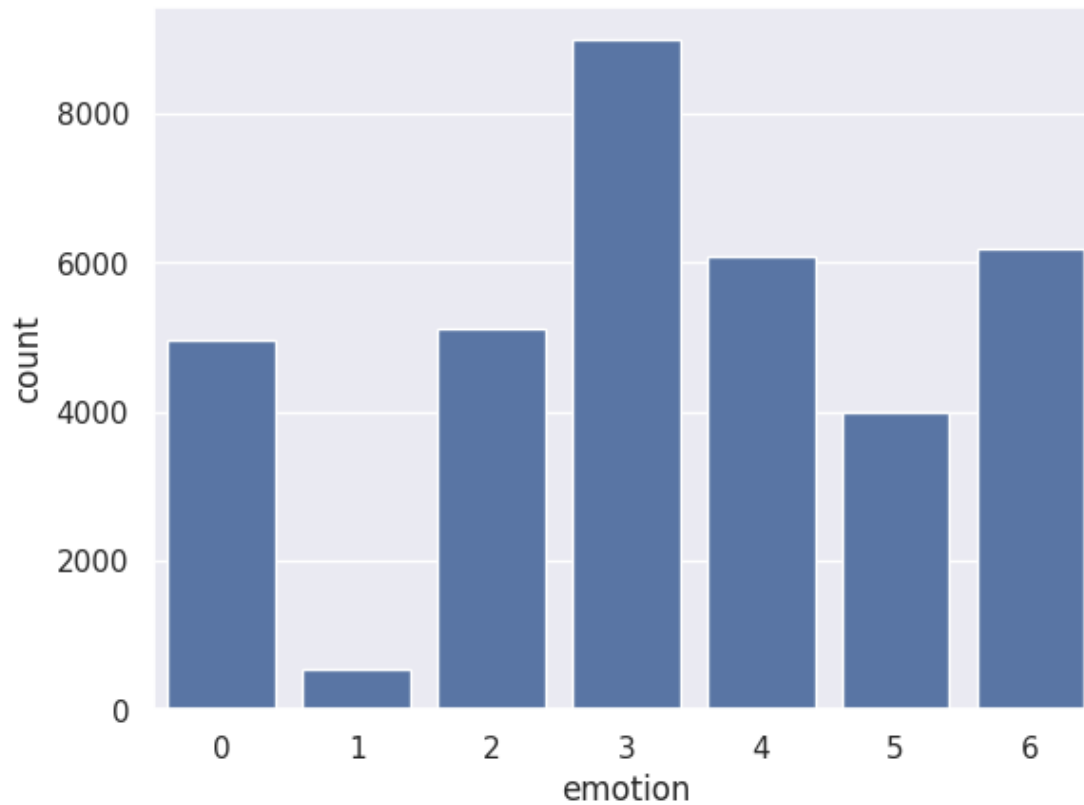
```
[63]: array([0, 2, 4, 6, 3, 5, 1])
```

```
[64]: emotion_mapping = {0: 'anger', 1: 'disgust', 2: 'fear', 3: 'happiness', 4: '
      ↪sadness', 5: 'surprise', 6: 'neutral'}
```

```
[65]: data_frame.emotion.value_counts()
```

```
[65]: emotion
3      8989
6      6198
4      6077
2      5121
0      4953
5      4002
1       547
Name: count, dtype: int64
```

```
[66]: sns.countplot(x=data_frame.emotion)
      pyplot.show()
```



```
[67]: import math

math.sqrt(len(data_frame.pixels[0].split(' ')))
```

[67]: 48.0

```
[68]: fig = pyplot.figure(1, (14, 14))

k = 0
for label in sorted(data_frame.emotion.unique()):
    for j in range(7):
        px = data_frame[data_frame.emotion == label].pixels.iloc[k]
        px = np.array(px.split(' ')).reshape(48, 48).astype('float32')

        k += 1
        ax = pyplot.subplot(7, 7, k)
        ax.imshow(px, cmap='gray')
        ax.set_xticks([])
        ax.set_yticks([])
        ax.set_title(emotion_mapping[label])
    pyplot.tight_layout()
```



```
[69]: selected_emotion_labels = [3, 4, 6]
```

```
[70]: data_frame = data_frame[data_frame.emotion.isin(selected_emotion_labels)]
data_frame.shape
```

```
[70]: (21264, 3)
```

```
[71]: img_array = data_frame.pixels.apply(lambda x: np.array(x.split(' ')).
↳ reshape(48, 48, 1).astype('float32'))
img_array = np.stack(img_array, axis=0)
```

```
[72]: img_array.shape
```

[72]: (21264, 48, 48, 1)

```
[73]: from tensorflow.keras.utils import to_categorical
      from sklearn.preprocessing import LabelEncoder

      le = LabelEncoder()
      img_labels = le.fit_transform(data_frame.emotion)
      img_labels = to_categorical(img_labels) # Use to_categorical from tensorflow.
      ↪keras.utils
      img_labels.shape
```

[73]: (21264, 3)

```
[74]: le_name_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
      print(le_name_mapping)

      {np.int64(3): np.int64(0), np.int64(4): np.int64(1), np.int64(6): np.int64(2)}
```

```
[75]: X_train, X_valid, y_train, y_valid = train_test_split(img_array, img_labels,
      shuffle=True, ↪
      ↪stratify=img_labels,
      test_size=0.1, ↪
      ↪random_state=42)
      X_train.shape, X_valid.shape, y_train.shape, y_valid.shape
```

[75]: ((19137, 48, 48, 1), (2127, 48, 48, 1), (19137, 3), (2127, 3))

```
[76]: del data_frame
      del img_array
      del img_labels
```

```
[77]: img_width = X_train.shape[1]
      img_height = X_train.shape[2]
      img_depth = X_train.shape[3]
      num_classes = y_train.shape[1]
```

```
[78]: # Normalizing results, as neural networks are very sensitive to unnormalized ↪
      ↪data.
      X_train = X_train / 255.
      X_valid = X_valid / 255.
```

```
[79]: def build_net(optim):
      """
      This is a Deep Convolutional Neural Network (DCNN). For generalization ↪
      ↪purpose I used dropouts in regular intervals.
      I used 'ELU' as the activation because it avoids dying relu problem but ↪
      ↪also performed well as compared to LeakyRelu
```

atleast in this case. `he_normal` kernel initializer is used as it suits `ELU`. `BatchNormalization` is also used for better

results.

"""

```
net = Sequential(name='DCNN')

net.add(
    Conv2D(
        filters=64,
        kernel_size=(5,5),
        input_shape=(img_width, img_height, img_depth),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_1'
    )
)

net.add(BatchNormalization(name='batchnorm_1'))
net.add(
    Conv2D(
        filters=64,
        kernel_size=(5,5),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_2'
    )
)

net.add(BatchNormalization(name='batchnorm_2'))

net.add(MaxPooling2D(pool_size=(2,2), name='maxpool2d_1'))
net.add(Dropout(0.4, name='dropout_1'))

net.add(
    Conv2D(
        filters=128,
        kernel_size=(3,3),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_3'
    )
)

net.add(BatchNormalization(name='batchnorm_3'))
net.add(
    Conv2D(
        filters=128,
```



```

        kernel_size=(3,3),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_4'
    )
)
net.add(BatchNormalization(name='batchnorm_4'))

net.add(MaxPooling2D(pool_size=(2,2), name='maxpool2d_2'))
net.add(Dropout(0.4, name='dropout_2'))

net.add(
    Conv2D(
        filters=256,
        kernel_size=(3,3),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_5'
    )
)
net.add(BatchNormalization(name='batchnorm_5'))
net.add(
    Conv2D(
        filters=256,
        kernel_size=(3,3),
        activation='elu',
        padding='same',
        kernel_initializer='he_normal',
        name='conv2d_6'
    )
)
net.add(BatchNormalization(name='batchnorm_6'))

net.add(MaxPooling2D(pool_size=(2,2), name='maxpool2d_3'))
net.add(Dropout(0.5, name='dropout_3'))

net.add(Flatten(name='flatten'))

net.add(
    Dense(
        128,
        activation='elu',
        kernel_initializer='he_normal',
        name='dense_1'
    )
)

```

```

)
net.add(BatchNormalization(name='batchnorm_7'))

net.add(Dropout(0.6, name='dropout_4'))

net.add(
    Dense(
        num_classes,
        activation='softmax',
        name='out_layer'
    )
)

net.compile(
    loss='categorical_crossentropy',
    optimizer=optim,
    metrics=['accuracy']
)

net.summary()

return net

```

[80]: *"""*
I used two callbacks one is `early stopping` for avoiding overfitting training_
↳data
and other `ReduceLROnPlateau` for learning rate.
"""

```

early_stopping = EarlyStopping(
    monitor='val_accuracy',
    min_delta=0.00005,
    patience=11,
    verbose=1,
    restore_best_weights=True,
)

lr_scheduler = ReduceLROnPlateau(
    monitor='val_accuracy',
    factor=0.5,
    patience=7,
    min_lr=1e-7,
    verbose=1,
)

callbacks = [
    early_stopping,

```

```
lr_scheduler,
]
```

[81]: *# As the data in hand is less as compared to the task so ImageDataGenerator is good to go.*

```
train_datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.15,
    height_shift_range=0.15,
    shear_range=0.15,
    zoom_range=0.15,
    horizontal_flip=True,
)
train_datagen.fit(X_train)
```

[27]: *batch_size = 32 #batch size of 32 performs the best.*

```
epochs = 100
optimizers = [
    optimizers.Nadam(learning_rate=0.001, beta_1=0.9, beta_2=0.999,
    epsilon=1e-07, name='Nadam'),
    optimizers.Adam(0.001),
]
```

I tried both `Nadam` and `Adam`, the difference in results is not different but I finally went with Nadam as it is more popular.

```
model = build_net(optimizers[1])
# Use model.fit instead of model.fit_generator
history = model.fit(
    train_datagen.flow(X_train, y_train, batch_size=batch_size),
    validation_data=(X_valid, y_valid),
    steps_per_epoch=len(X_train) // batch_size, # Changed to floor division
    epochs=epochs,
    callbacks=callbacks,
    # use_multiprocessing=True # Remove this line if it causes issues
)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "DCNN"
```

Layer (type)

Output Shape

Param #

conv2d_1 (Conv2D)	(None, 48, 48, 64)	1,664
batchnorm_1 (BatchNormalization)	(None, 48, 48, 64)	256
conv2d_2 (Conv2D)	(None, 48, 48, 64)	102,464
batchnorm_2 (BatchNormalization)	(None, 48, 48, 64)	256
maxpool2d_1 (MaxPooling2D)	(None, 24, 24, 64)	0
dropout_1 (Dropout)	(None, 24, 24, 64)	0
conv2d_3 (Conv2D)	(None, 24, 24, 128)	73,856
batchnorm_3 (BatchNormalization)	(None, 24, 24, 128)	512
conv2d_4 (Conv2D)	(None, 24, 24, 128)	147,584
batchnorm_4 (BatchNormalization)	(None, 24, 24, 128)	512
maxpool2d_2 (MaxPooling2D)	(None, 12, 12, 128)	0
dropout_2 (Dropout)	(None, 12, 12, 128)	0
conv2d_5 (Conv2D)	(None, 12, 12, 256)	295,168
batchnorm_5 (BatchNormalization)	(None, 12, 12, 256)	1,024
conv2d_6 (Conv2D)	(None, 12, 12, 256)	590,080
batchnorm_6 (BatchNormalization)	(None, 12, 12, 256)	1,024
maxpool2d_3 (MaxPooling2D)	(None, 6, 6, 256)	0
dropout_3 (Dropout)	(None, 6, 6, 256)	0
flatten (Flatten)	(None, 9216)	0
dense_1 (Dense)	(None, 128)	1,179,776
batchnorm_7	(None, 128)	512

(BatchNormalization)

dropout_4 (Dropout) (None, 128) 0

out_layer (Dense) (None, 3) 387

Total params: 2,395,075 (9.14 MB)

Trainable params: 2,393,027 (9.13 MB)

Non-trainable params: 2,048 (8.00 KB)

Epoch 1/100

/usr/local/lib/python3.11/dist-

packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121:

UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

self._warn_if_super_not_called()

598/598 175s 273ms/step -

accuracy: 0.3691 - loss: 1.5908 - val_accuracy: 0.5346 - val_loss: 0.9802 -

learning_rate: 0.0010

Epoch 2/100

1/598 2:47 280ms/step -

accuracy: 0.3750 - loss: 1.2082

/usr/local/lib/python3.11/dist-

packages/keras/src/trainers/epoch_iterator.py:107: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.

self._interrupted_warning()

598/598 3s 4ms/step -

accuracy: 0.3750 - loss: 1.2082 - val_accuracy: 0.5369 - val_loss: 0.9794 -

learning_rate: 0.0010

Epoch 3/100

598/598 165s 276ms/step -

accuracy: 0.4752 - loss: 1.0374 - val_accuracy: 0.5679 - val_loss: 0.9046 -

learning_rate: 0.0010

Epoch 4/100

598/598 3s 4ms/step -

accuracy: 0.5312 - loss: 0.9133 - val_accuracy: 0.5599 - val_loss: 0.9097 -

learning_rate: 0.0010

Epoch 5/100
598/598 164s 275ms/step -
accuracy: 0.5488 - loss: 0.9309 - val_accuracy: 0.6540 - val_loss: 0.7662 -
learning_rate: 0.0010

Epoch 6/100
598/598 3s 4ms/step -
accuracy: 0.5625 - loss: 0.9125 - val_accuracy: 0.6474 - val_loss: 0.7670 -
learning_rate: 0.0010

Epoch 7/100
598/598 163s 272ms/step -
accuracy: 0.6084 - loss: 0.8277 - val_accuracy: 0.6436 - val_loss: 0.7652 -
learning_rate: 0.0010

Epoch 8/100
598/598 2s 3ms/step -
accuracy: 0.7188 - loss: 0.6855 - val_accuracy: 0.6436 - val_loss: 0.7666 -
learning_rate: 0.0010

Epoch 9/100
598/598 163s 273ms/step -
accuracy: 0.6468 - loss: 0.7639 - val_accuracy: 0.7076 - val_loss: 0.6532 -
learning_rate: 0.0010

Epoch 10/100
598/598 2s 4ms/step -
accuracy: 0.5938 - loss: 0.7543 - val_accuracy: 0.7132 - val_loss: 0.6498 -
learning_rate: 0.0010

Epoch 11/100
598/598 162s 272ms/step -
accuracy: 0.6651 - loss: 0.7220 - val_accuracy: 0.7348 - val_loss: 0.6219 -
learning_rate: 0.0010

Epoch 12/100
598/598 2s 3ms/step -
accuracy: 0.6562 - loss: 0.6487 - val_accuracy: 0.7367 - val_loss: 0.6203 -
learning_rate: 0.0010

Epoch 13/100
598/598 161s 270ms/step -
accuracy: 0.6878 - loss: 0.6947 - val_accuracy: 0.7400 - val_loss: 0.6122 -
learning_rate: 0.0010

Epoch 14/100
598/598 2s 4ms/step -
accuracy: 0.7500 - loss: 0.7717 - val_accuracy: 0.7400 - val_loss: 0.6136 -
learning_rate: 0.0010

Epoch 15/100
598/598 161s 269ms/step -
accuracy: 0.6999 - loss: 0.6706 - val_accuracy: 0.7409 - val_loss: 0.5983 -
learning_rate: 0.0010

Epoch 16/100
598/598 2s 3ms/step -
accuracy: 0.6250 - loss: 0.7362 - val_accuracy: 0.7381 - val_loss: 0.5977 -
learning_rate: 0.0010

Epoch 17/100
598/598 162s 272ms/step -
accuracy: 0.7119 - loss: 0.6548 - val_accuracy: 0.7569 - val_loss: 0.5761 -
learning_rate: 0.0010
Epoch 18/100
598/598 2s 3ms/step -
accuracy: 0.6875 - loss: 0.6276 - val_accuracy: 0.7593 - val_loss: 0.5741 -
learning_rate: 0.0010
Epoch 19/100
598/598 162s 271ms/step -
accuracy: 0.7238 - loss: 0.6320 - val_accuracy: 0.7466 - val_loss: 0.5838 -
learning_rate: 0.0010
Epoch 20/100
598/598 2s 4ms/step -
accuracy: 0.6562 - loss: 0.7126 - val_accuracy: 0.7457 - val_loss: 0.5882 -
learning_rate: 0.0010
Epoch 21/100
598/598 161s 270ms/step -
accuracy: 0.7300 - loss: 0.6290 - val_accuracy: 0.7659 - val_loss: 0.5523 -
learning_rate: 0.0010
Epoch 22/100
598/598 2s 4ms/step -
accuracy: 0.8125 - loss: 0.5342 - val_accuracy: 0.7649 - val_loss: 0.5511 -
learning_rate: 0.0010
Epoch 23/100
598/598 161s 270ms/step -
accuracy: 0.7404 - loss: 0.6089 - val_accuracy: 0.7438 - val_loss: 0.5718 -
learning_rate: 0.0010
Epoch 24/100
598/598 2s 4ms/step -
accuracy: 0.8750 - loss: 0.5689 - val_accuracy: 0.7485 - val_loss: 0.5675 -
learning_rate: 0.0010
Epoch 25/100
598/598 162s 271ms/step -
accuracy: 0.7408 - loss: 0.5993 - val_accuracy: 0.7870 - val_loss: 0.5139 -
learning_rate: 0.0010
Epoch 26/100
598/598 2s 4ms/step -
accuracy: 0.6875 - loss: 0.6355 - val_accuracy: 0.7880 - val_loss: 0.5155 -
learning_rate: 0.0010
Epoch 27/100
598/598 163s 272ms/step -
accuracy: 0.7461 - loss: 0.5856 - val_accuracy: 0.7856 - val_loss: 0.5149 -
learning_rate: 0.0010
Epoch 28/100
598/598 2s 4ms/step -
accuracy: 0.8438 - loss: 0.5499 - val_accuracy: 0.7837 - val_loss: 0.5144 -
learning_rate: 0.0010

Epoch 29/100
598/598 162s 271ms/step -
accuracy: 0.7472 - loss: 0.5871 - val_accuracy: 0.7776 - val_loss: 0.5357 -
learning_rate: 0.0010

Epoch 30/100
598/598 2s 3ms/step -
accuracy: 0.7500 - loss: 0.5127 - val_accuracy: 0.7776 - val_loss: 0.5412 -
learning_rate: 0.0010

Epoch 31/100
598/598 162s 271ms/step -
accuracy: 0.7581 - loss: 0.5739 - val_accuracy: 0.7978 - val_loss: 0.5077 -
learning_rate: 0.0010

Epoch 32/100
598/598 2s 4ms/step -
accuracy: 0.7500 - loss: 0.5661 - val_accuracy: 0.7931 - val_loss: 0.5119 -
learning_rate: 0.0010

Epoch 33/100
598/598 161s 269ms/step -
accuracy: 0.7576 - loss: 0.5674 - val_accuracy: 0.7847 - val_loss: 0.5157 -
learning_rate: 0.0010

Epoch 34/100
598/598 2s 4ms/step -
accuracy: 0.6875 - loss: 0.6028 - val_accuracy: 0.7861 - val_loss: 0.5175 -
learning_rate: 0.0010

Epoch 35/100
598/598 161s 269ms/step -
accuracy: 0.7556 - loss: 0.5671 - val_accuracy: 0.7992 - val_loss: 0.4937 -
learning_rate: 0.0010

Epoch 36/100
598/598 2s 3ms/step -
accuracy: 0.7188 - loss: 0.5605 - val_accuracy: 0.7969 - val_loss: 0.4955 -
learning_rate: 0.0010

Epoch 37/100
598/598 162s 271ms/step -
accuracy: 0.7664 - loss: 0.5543 - val_accuracy: 0.7969 - val_loss: 0.5021 -
learning_rate: 0.0010

Epoch 38/100
598/598 2s 3ms/step -
accuracy: 0.8438 - loss: 0.3936 - val_accuracy: 0.7974 - val_loss: 0.5020 -
learning_rate: 0.0010

Epoch 39/100
598/598 162s 270ms/step -
accuracy: 0.7750 - loss: 0.5423 - val_accuracy: 0.7974 - val_loss: 0.4961 -
learning_rate: 0.0010

Epoch 40/100
598/598 2s 3ms/step -
accuracy: 0.8125 - loss: 0.4892 - val_accuracy: 0.7978 - val_loss: 0.4967 -
learning_rate: 0.0010

Epoch 41/100
598/598 163s 272ms/step -
accuracy: 0.7678 - loss: 0.5411 - val_accuracy: 0.7955 - val_loss: 0.5013 -
learning_rate: 0.0010

Epoch 42/100
1/598 2:32 255ms/step -
accuracy: 0.7188 - loss: 0.7150
Epoch 42: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
598/598 2s 3ms/step -
accuracy: 0.7188 - loss: 0.7150 - val_accuracy: 0.7913 - val_loss: 0.5041 -
learning_rate: 0.0010

Epoch 43/100
598/598 161s 269ms/step -
accuracy: 0.7727 - loss: 0.5395 - val_accuracy: 0.8087 - val_loss: 0.4758 -
learning_rate: 5.0000e-04

Epoch 44/100
598/598 2s 3ms/step -
accuracy: 0.7812 - loss: 0.5853 - val_accuracy: 0.8091 - val_loss: 0.4759 -
learning_rate: 5.0000e-04

Epoch 45/100
598/598 162s 270ms/step -
accuracy: 0.7842 - loss: 0.5085 - val_accuracy: 0.8007 - val_loss: 0.4892 -
learning_rate: 5.0000e-04

Epoch 46/100
598/598 2s 4ms/step -
accuracy: 0.7812 - loss: 0.5353 - val_accuracy: 0.7997 - val_loss: 0.4902 -
learning_rate: 5.0000e-04

Epoch 47/100
598/598 162s 271ms/step -
accuracy: 0.7922 - loss: 0.5121 - val_accuracy: 0.8115 - val_loss: 0.4725 -
learning_rate: 5.0000e-04

Epoch 48/100
598/598 2s 4ms/step -
accuracy: 0.9062 - loss: 0.4012 - val_accuracy: 0.8115 - val_loss: 0.4734 -
learning_rate: 5.0000e-04

Epoch 49/100
598/598 162s 270ms/step -
accuracy: 0.7955 - loss: 0.4953 - val_accuracy: 0.8035 - val_loss: 0.4841 -
learning_rate: 5.0000e-04

Epoch 50/100
598/598 2s 3ms/step -
accuracy: 0.8125 - loss: 0.3170 - val_accuracy: 0.8044 - val_loss: 0.4829 -
learning_rate: 5.0000e-04

Epoch 51/100
598/598 162s 270ms/step -
accuracy: 0.7935 - loss: 0.4926 - val_accuracy: 0.8030 - val_loss: 0.4732 -
learning_rate: 5.0000e-04

Epoch 52/100

598/598 2s 4ms/step -
accuracy: 0.8750 - loss: 0.4223 - val_accuracy: 0.8030 - val_loss: 0.4734 -
learning_rate: 5.0000e-04
Epoch 53/100

598/598 162s 271ms/step -
accuracy: 0.7995 - loss: 0.4940 - val_accuracy: 0.8011 - val_loss: 0.4912 -
learning_rate: 5.0000e-04
Epoch 54/100

1/598 2:41 270ms/step -
accuracy: 0.7812 - loss: 0.7095
Epoch 54: ReduceLR0nPlateau reducing learning rate to 0.0002500000118743628.

598/598 2s 3ms/step -
accuracy: 0.7812 - loss: 0.7095 - val_accuracy: 0.8030 - val_loss: 0.4930 -
learning_rate: 5.0000e-04
Epoch 55/100

598/598 162s 271ms/step -
accuracy: 0.8027 - loss: 0.4799 - val_accuracy: 0.8181 - val_loss: 0.4621 -
learning_rate: 2.5000e-04
Epoch 56/100

598/598 2s 3ms/step -
accuracy: 0.8750 - loss: 0.4121 - val_accuracy: 0.8171 - val_loss: 0.4614 -
learning_rate: 2.5000e-04
Epoch 57/100

598/598 161s 269ms/step -
accuracy: 0.8053 - loss: 0.4697 - val_accuracy: 0.8171 - val_loss: 0.4697 -
learning_rate: 2.5000e-04
Epoch 58/100

598/598 2s 3ms/step -
accuracy: 0.7188 - loss: 0.6379 - val_accuracy: 0.8166 - val_loss: 0.4693 -
learning_rate: 2.5000e-04
Epoch 59/100

598/598 161s 269ms/step -
accuracy: 0.8045 - loss: 0.4772 - val_accuracy: 0.8152 - val_loss: 0.4681 -
learning_rate: 2.5000e-04
Epoch 60/100

598/598 2s 4ms/step -
accuracy: 0.7812 - loss: 0.4491 - val_accuracy: 0.8152 - val_loss: 0.4679 -
learning_rate: 2.5000e-04
Epoch 61/100

598/598 161s 268ms/step -
accuracy: 0.8049 - loss: 0.4730 - val_accuracy: 0.8115 - val_loss: 0.4674 -
learning_rate: 2.5000e-04
Epoch 62/100

1/598 2:44 275ms/step -
accuracy: 0.8125 - loss: 0.4167
Epoch 62: ReduceLR0nPlateau reducing learning rate to 0.0001250000059371814.

598/598 2s 4ms/step -
accuracy: 0.8125 - loss: 0.4167 - val_accuracy: 0.8115 - val_loss: 0.4671 -

```

learning_rate: 2.5000e-04
Epoch 63/100
598/598          161s 269ms/step -
accuracy: 0.8049 - loss: 0.4672 - val_accuracy: 0.8134 - val_loss: 0.4618 -
learning_rate: 1.2500e-04
Epoch 64/100
598/598          2s 4ms/step -
accuracy: 0.8125 - loss: 0.3597 - val_accuracy: 0.8129 - val_loss: 0.4618 -
learning_rate: 1.2500e-04
Epoch 65/100
598/598          161s 269ms/step -
accuracy: 0.8050 - loss: 0.4599 - val_accuracy: 0.8152 - val_loss: 0.4645 -
learning_rate: 1.2500e-04
Epoch 66/100
598/598          2s 3ms/step -
accuracy: 0.9062 - loss: 0.3086 - val_accuracy: 0.8157 - val_loss: 0.4645 -
learning_rate: 1.2500e-04
Epoch 66: early stopping
Restoring model weights from the end of the best epoch: 55.

```

```

[82]: import tensorflow as tf

# Convert the model to JSON format
model_json = model.to_json() # Use to_json instead of to_yaml

# Save the JSON to a file
with open("model.json", "w") as json_file:
    json_file.write(model_json)

# Save the model weights and architecture
model.save("model.h5")

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

[83]: sns.set()
fig = pyplot.figure(0, (12, 4))

ax = pyplot.subplot(1, 2, 1)
# Pass x and y as keyword arguments
sns.lineplot(x=history.epoch, y=history.history['accuracy'], label='train')
sns.lineplot(x=history.epoch, y=history.history['val_accuracy'], label='valid')
pyplot.title('Accuracy')
pyplot.tight_layout()

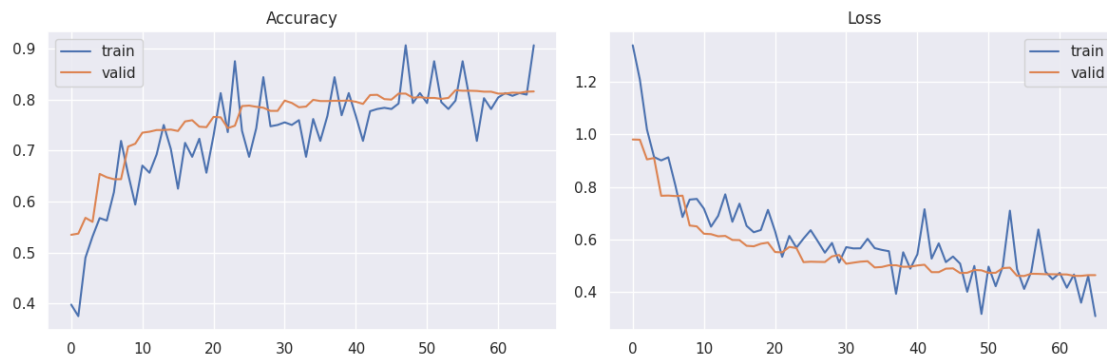
```

```

ax = pyplot.subplot(1, 2, 2)
# Pass x and y as keyword arguments
sns.lineplot(x=history.epoch, y=history.history['loss'], label='train')
sns.lineplot(x=history.epoch, y=history.history['val_loss'], label='valid')
pyplot.title('Loss')
pyplot.tight_layout()

pyplot.savefig('epoch_history_dcnm.png')
pyplot.show()

```



```

[84]: import pandas as pd
import seaborn as sns
from matplotlib import pyplot

# Creating DataFrames
df_accu = pd.DataFrame({
    'train': history.history['accuracy'],
    'valid': history.history['val_accuracy']
})
df_loss = pd.DataFrame({
    'train': history.history['loss'],
    'valid': history.history['val_loss']
})

# Plotting
fig = pyplot.figure(0, (14, 4))

# Accuracy plot
ax = pyplot.subplot(1, 2, 1)
sns.violinplot(x="variable", y="value", data=pd.melt(df_accu)) # REMOVED
    ↪ showfliers
pyplot.title('Accuracy')
pyplot.tight_layout()

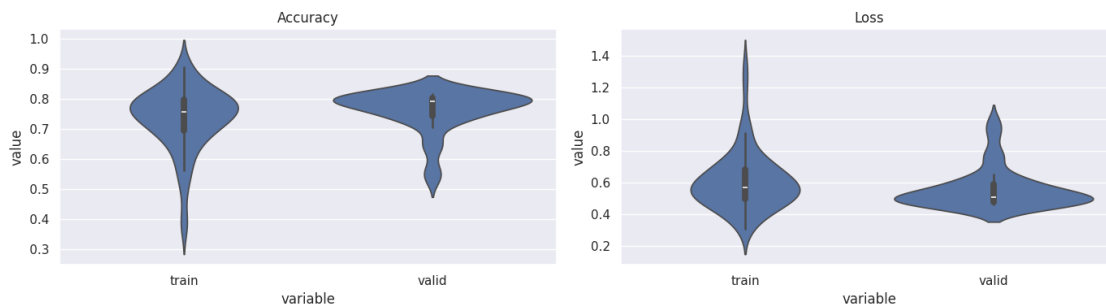
```

```

# Loss plot
ax = pyplot.subplot(1, 2, 2)
sns.violinplot(x="variable", y="value", data=pd.melt(df_loss)) # REMOVED
    ↪ showliers
pyplot.title('Loss')
pyplot.tight_layout()

# Save and show
pyplot.savefig('performance_dist.png')
pyplot.show()

```



```

[85]: from sklearn.metrics import classification_report, confusion_matrix,
    ↪ ConfusionMatrixDisplay

# Predict probabilities
yhat_valid_probs = model.predict(X_valid)

# Take class with highest probability
yhat_valid = np.argmax(yhat_valid_probs, axis=1)

# Plot confusion matrix manually using sklearn
cm = confusion_matrix(np.argmax(y_valid, axis=1), yhat_valid)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', values_format='d', xticks_rotation=45)
pyplot.title('Confusion Matrix')
pyplot.savefig("confusion_matrix_dcnn.png")
pyplot.show()

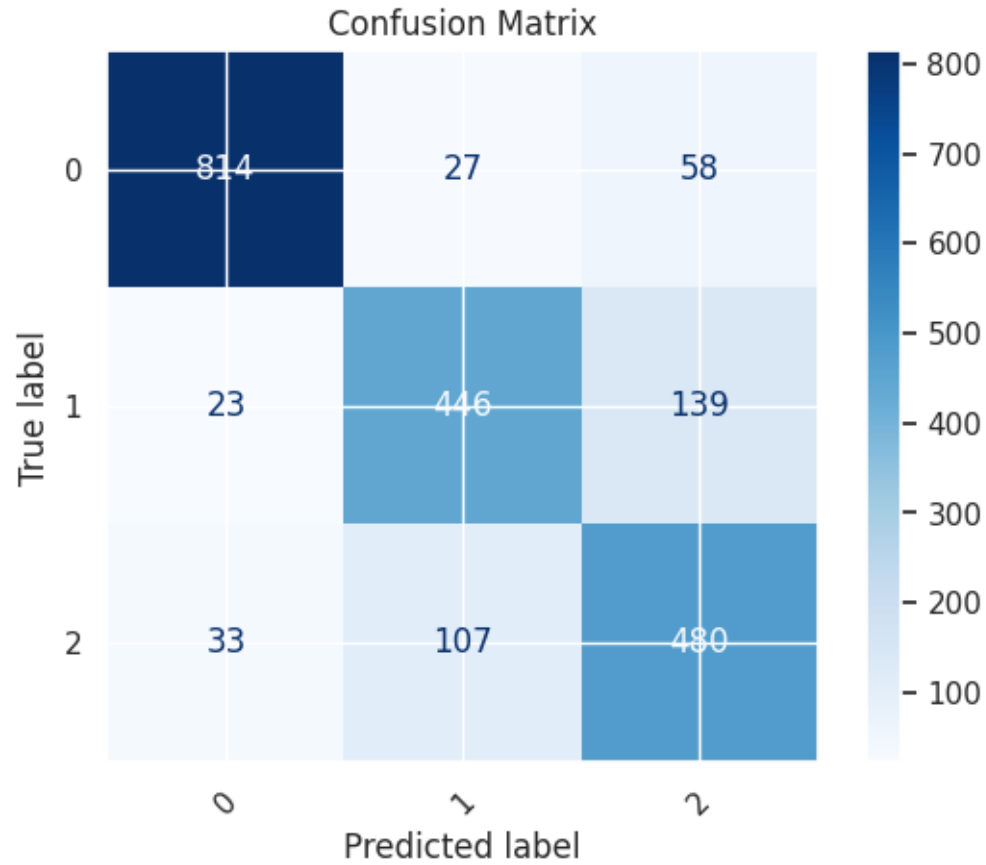
# Print wrong predictions
print(f'Total wrong validation predictions: {np.sum(np.argmax(y_valid, axis=1) !=
    ↪ yhat_valid)}\n\n')

# Print classification report
print(classification_report(np.argmax(y_valid, axis=1), yhat_valid))

```

67/67

2s 31ms/step



Total wrong validation predictions: 387

	precision	recall	f1-score	support
0	0.94	0.91	0.92	899
1	0.77	0.73	0.75	608
2	0.71	0.77	0.74	620
accuracy			0.82	2127
macro avg	0.80	0.80	0.80	2127
weighted avg	0.82	0.82	0.82	2127

```
[86]: mapper = {
      0: "happy",
      1: "sad",
      2: "neutral",
```

```
}
```

```
[87]: np.random.seed(2)

# Pick random indices
random_sad_imgs = np.random.choice(np.where(y_valid[:, 1] == 1)[0], size=9)
random_neutral_imgs = np.random.choice(np.where(y_valid[:, 2] == 1)[0], size=9)

# Create figure
fig = pyplot.figure(1, (18, 4))

for i, (sadidx, neuidx) in enumerate(zip(random_sad_imgs, random_neutral_imgs)):
    # Sad images
    ax = pyplot.subplot(2, 9, i+1)
    sample_img = X_valid[sadidx, :, :, 0]
    ax.imshow(sample_img, cmap='gray')
    ax.set_xticks([])
    ax.set_yticks([])

    # Predict class correctly
    pred_class_sad = np.argmax(model.predict(sample_img.reshape(1, 48, 48, 1)),
    ↪axis=1)[0]
    ax.set_title(f"true:sad, pred:{mapper[pred_class_sad]}")

    # Neutral images
    ax = pyplot.subplot(2, 9, i+10)
    sample_img = X_valid[neuidx, :, :, 0]
    ax.imshow(sample_img, cmap='gray')
    ax.set_xticks([])
    ax.set_yticks([])

    # Predict class correctly
    pred_class_neu = np.argmax(model.predict(sample_img.reshape(1, 48, 48, 1)),
    ↪axis=1)[0]
    ax.set_title(f"t:neut, p:{mapper[pred_class_neu]}")

pyplot.tight_layout()
pyplot.show()
```

```
1/1          0s 50ms/step
1/1          0s 47ms/step
1/1          0s 48ms/step
1/1          0s 47ms/step
1/1          0s 45ms/step
1/1          0s 48ms/step
1/1          0s 47ms/step
1/1          0s 48ms/step
```

1/1	0s 48ms/step
1/1	0s 55ms/step
1/1	0s 49ms/step
1/1	0s 47ms/step
1/1	0s 48ms/step
1/1	0s 48ms/step
1/1	0s 46ms/step
1/1	0s 46ms/step
1/1	0s 47ms/step
1/1	0s 47ms/step

